TUTOR MATCH – A CLOUD-BASED PLATFORM FOR STUDENT-TUTOR DISCOVERY AND BOOKING

CLOUD COMPUTING

Submitted by

SANJANA R (230701285)
SRE VARSHA N (230701330)
NIRANJANA K (230701400)

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING RAJALAKSHMI

ENGINEERING COLLEGE

MAY 2026

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this project report titled "**Tutor Match – A Cloud-Based Platform for Student-Tutor Discovery and Booking** " is the bonafide work of **SANJANA R (230701285), SRE VARSHA N (230701330), NIRANJANA K (230701400)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. E. M Malathy
Professor &
Head of The Department
Department of Computer Science
and Engineering
Rajalakshmi Engineering College

SIGNATURE

Dr.M.Ayyadurai
Supervisor
Associate Professor
Department of Computer Science
and Engineering
Rajalakshmi Engineering College

Submitted to Project Viva Voce Examination held on_____

Internal Examiner **External Examiner**

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S.MEGANATHAN, B.E, F.I.E.**, our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.**, and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.,** for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.**, our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr.MALATHY E.M, Ph.D.**, Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, **DR. M. AYYADURAI M.E.,Ph.D.,** Associate Professor, Department of Computer Science and Engineering for his valuable guidance throughout the course of the project.

ABSTRACT

Students often face difficulty in finding qualified tutors for specific subjects due to the absence of a centralized and reliable platform that enables easy tutor discovery and booking. Traditional methods of searching for tutors through social media, messaging groups, or personal referrals are time-consuming, unorganized, and lack transparency regarding tutor qualifications, availability, ratings, and pricing. Studies show that over 55% of students depend on informal sources to find academic guidance, while nearly 40% struggle to connect with suitable tutors within their preferred schedule and budget. Existing educational platforms mainly focus on large-scale online coaching or prerecorded courses rather than providing a direct and efficient connection between students and individual tutors. This creates challenges in personalized learning, communication, and session management. The proposed project aims to develop a secure and scalable cloud-based tutor-student matching and booking platform where students can browse tutor profiles, view subject expertise, experience, ratings, availability, and book tutoring sessions based on their academic requirements. The platform provides role-based access for students and tutors, secure authentication mechanisms, centralized profile management, session scheduling, and cloud-based data storage for handling user information and booking records efficiently. The system architecture is designed using a modular and service-oriented approach to ensure scalability, high availability, and optimized request handling for concurrent users. Backend services manage authentication, booking workflows, tutor profile management, and session data processing, while the frontend offers an intuitive and responsive interface for seamless interaction between users. Secure database integration ensures reliable storage of tutor details, student records, schedules, and booking history, while automated backend operations improve system responsiveness and reduce manual overhead. The expected outcome of the project is an efficient, reliable, and user-friendly tutoring platform that simplifies the process of connecting students with suitable tutors, improves accessibility to personalized academic support, and enhances the overall tutoring experience through secure and scalable cloud technologies.

TABLE OF CONTENTS

CHAPTER 1 - INTRODUCTION

1.1 PROBLEM STATEMENT

Students today often rely on fragmented and unreliable methods to find qualified tutors for academic support. Traditional approaches such as social media groups, messaging platforms, coaching advertisements, and personal referrals present several challenges in terms of accessibility, trust, scalability, and efficiency:

- Difficulty in Finding Suitable Tutors: Students frequently struggle to identify tutors who match their academic requirements, preferred schedules, learning pace, and budget. Studies indicate that more than 55% of students depend on informal sources to find tutors, resulting in time-consuming searches and inconsistent tutoring experiences.

- Lack of Trust and Verification: Existing tutor discovery methods provide limited transparency regarding tutor qualifications, certifications, teaching experience, and credibility. Many students face concerns related to fake profiles, misleading claims, and unverified credentials, reducing confidence in online tutor selection.

- Inefficient Booking and Communication: Traditional systems lack centralized mechanisms for tutor profile management, availability tracking, and session booking. Students and tutors often rely on manual communication methods, leading to scheduling conflicts, delayed responses, and poor user experience.

- Limited Personalization in Existing Platforms: Most current educational platforms focus on large-scale coaching classes or prerecorded content rather than enabling direct and personalized tutor–student interaction. This limits students from accessing one-to-one academic guidance tailored to their individual learning needs.

- Administrative and Scalability Challenges: Manual tutor approval and credential verification processes increase administrative workload and reduce operational efficiency. As the number of tutor registrations increases, verifying uploaded certificates and maintaining platform authenticity becomes increasingly difficult without intelligent automation.

  Combined, these issues result in poor accessibility to personalized academic support, inefficient tutor discovery processes, lack of trust in tutor authenticity, and reduced overall effectiveness of digital tutoring platforms.

1.2 OBJECTIVE OF THE PROJECT

The primary objective of this project is to design and develop a cloud-based tutor discovery and booking platform that enables students to efficiently connect with qualified tutors based on subject expertise, availability, and academic requirements. The system aims to simplify tutor onboarding, automate verification workflows, and provide a secure, scalable, and user-friendly environment for tutor–student interaction.

1. **Simplify Tutor Discovery and Booking**

   Develop a centralized platform where students can easily search and discover tutors based on subject specialization, ratings, teaching experience, and availability. The system allows students to compare tutor profiles and book tutoring sessions efficiently through an intuitive and user-friendly interface.

2. **Implement Secure Authentication and Access Control**

   Integrate secure authentication using Microsoft Azure App Registration to provide safe login functionality for users. The system also ensures role-based access control for students, tutors, and administrators to maintain security and proper authorization.

3. **Automate Tutor Verification Using AI**

   Implement AI-assisted tutor approval workflows using Microsoft Azure OpenAI services to validate uploaded tutor certificates and profile information. The platform automatically classifies tutor applications into approved, rejected, or admin-review categories based on verification confidence scores and analysis results.

4. **Enhance User Interaction and Academic Assistance**

   Provide an AI-powered chatbot that assists students with platform navigation, tutor discovery, booking guidance, and academic-related queries. This improves user engagement, accessibility, and overall user experience within the platform.

5. **Ensure Scalable and Reliable Cloud Deployment**

   Utilize Microsoft Azure cloud infrastructure for hosting frontend applications, backend APIs, authentication systems, database services, and blob storage. This ensures scalability, reliability, high availability, and efficient resource management.

6. **Enable Efficient Data and Booking Management**

   Maintain centralized storage of tutor profiles, student information, booking records, uploaded certificates, and tutoring session details using a structured relational database system. This helps ensure organized data handling, quick retrieval, and efficient management of platform operations.

1.3 SCOPE AND BOUNDARIES

The scope of this project encompasses the complete development and deployment of a cloud-based tutor discovery and booking web application integrated with AI-powered automation and Azure cloud infrastructure. This includes secure user authentication, tutor profile management, session booking workflows, AI-assisted tutor verification, chatbot integration, cloud storage management, and automated deployment pipelines.The system supports core platform functionalities such as student and tutor registration, Microsoft-based authentication, tutor profile creation, certificate uploads, tutor search and filtering, booking management, and AI-powered chatbot assistance. The application also incorporates an intelligent tutor approval workflow where uploaded tutor certificates are analyzed using Azure OpenAI models for fake document detection and authenticity validation. Based on AI confidence scores, tutor applications are automatically approved, rejected, or forwarded for manual administrative review. The frontend application is hosted using Azure Static Web Apps, while backend APIs are implemented using Azure Functions with structured data persistence managed through Azure SQL Database and file storage handled using Azure Blob Storage.

Continuous integration and deployment are implemented using GitHub Actions to automate frontend deployment and backend update workflows. The current scope of the project is limited to tutor discovery, booking management, AI-based verification, and chatbot functionalities within a web-based platform environment. Features such as payment gateway integration, live video tutoring, advanced recommendation systems, and dedicated mobile applications are outside the present project scope and may be considered for future enhancements.

1.4 STAKEHOLDERS AND END USERS

Stakeholders:

Project Developers / Team Members: Responsible for designing, developing, deploying, and maintaining the frontend, backend APIs, AI workflows, database systems, and cloud infrastructure.

Platform Administrators: Manage tutor approvals, monitor platform activities, review AI-flagged tutor applications, and maintain platform integrity.

End Users:

Students: Students who want to search for qualified tutors, compare tutor profiles, and book academic tutoring sessions based on their learning requirements.

Tutors: Tutors who register on the platform, upload credentials, manage subject expertise and availability, and provide academic guidance to students.

Administrators: Authorized personnel responsible for managing tutor verification workflows, reviewing

flagged applications, and maintaining secure platform operations.

1.5 TECHNOLOGIES USED

The project incorporates a modern technology stack tailored for performance, scalability, and ease of development:

Table 1. Frontend Technologies

HTML	Structure and layout of the web application
CSS	Styling and responsive UI
JavaScript	Frontend interactivity and API integration

Table 2: Backend Technologies

Azure Functions	Serverless backend APIs and endpoint management
JavaScript (Node.js)	Backend logic and API handling
Azure SQL Database	Structured relational database storage
Blob Storage	Storage of uploaded tutor certificates and documents

Table 3: Cloud and Infrastructure

Microsoft Azure	The primary cloud provider for hosting and managing services.
Azure Container Apps	For container orchestration with auto-scaling.
Azure App Registration	Microsoft Authentication and secure login
Azure Storage Account	For blob storage of user-uploaded images.
Terraform	For IaaC provision
Docker	For containerizing applications.

Table 4: AI and Machine Learning

Azure OpenAI Services	For AI Chatbot integration and intelligence
Azure OpenAI Vision Models	For fake document detection

Table 5: DevOps and CI/CD

GitHub Actions	For automating workflows, builds, and deployments.
Docker Hub	As a registry for storing and distributing container images.
Azure CLI	For command-line management of Azure resources.

1.6 ORGANIZATION OF THE REPORT

This report is organized to provide a systematic and comprehensive overview of the entire project development lifecycle.

Chapter 1 introduces the project background, problem statement, objectives, scope, stakeholders, technologies used, and overall report structure.

Chapter 2 presents the system architecture and design, including frontend-backend interaction, Azure cloud integration, database structure, authentication workflow, and AI service architecture.

Chapter 3 discusses the implementation details of the project, including frontend development, Azure Function APIs, database integration, tutor booking workflows, and blob storage integration.

Chapter 4 explains the AI-powered functionalities and cloud deployment processes, including Azure OpenAI chatbot integration, tutor certificate verification workflows, CI/CD pipelines using GitHub Actions, and deployment using Azure services.

Chapter 5 presents the project results, testing outcomes, system screenshots, performance observations, challenges faced during implementation, and overall project evaluation.

Chapter 6 concludes the report with a summary of achievements, limitations of the current system, and future enhancement possibilities such as payment integration, mobile application support, and live tutoring functionalities.

The report concludes with references and appendices containing implementation details, API samples, database schemas, and deployment screenshots

CHAPTER 2 - SYSTEM DESIGN AND ARCHITECTURE

2.1 REQUIREMENT SUMMARY

Functional Requirements

The system consists of multiple cloud-based components deployed primarily on Microsoft Azure to support a scalable, secure, and intelligent tutor discovery and booking platform.

The frontend of the application is developed using HTML, CSS, and JavaScript and hosted on Azure Static Web Apps. This service provides reliable hosting, integrated GitHub connectivity, and automated CI/CD deployment workflows through GitHub Actions.

The backend logic is implemented using Azure Functions, where multiple serverless API endpoints handle functionalities such as user authentication, tutor profile management, tutor booking workflows, chatbot interaction, certificate verification handling, and database operations. Cross-Origin Resource Sharing (CORS) is configured to securely enable communication between frontend and backend services.

User information, tutor profiles, booking records, schedules, ratings, and credential verification details are stored in Azure SQL Database, which provides structured relational storage with secure cloud-based data persistence and scalable query handling.

To manage uploaded tutor certificates and related verification documents, the system uses Azure Blob Storage, enabling secure and efficient storage of images and files within the cloud infrastructure.

The application integrates Microsoft Authentication through Azure App Registration to provide secure login functionality and role-based access control for students, tutors, and administrators.

AI-assisted automation is implemented using Azure OpenAI services. Students can interact with an AI-powered chatbot for assistance and platform navigation. Additionally, tutor-uploaded certificates are analyzed using Azure OpenAI vision capabilities for fake document detection and authenticity validation. Based on AI-generated confidence scores, tutor applications are automatically approved, rejected, or forwarded for manual administrative review.

Continuous integration and deployment pipelines are automated using GitHub Actions integrated with Azure services, enabling seamless frontend deployment and backend updates automatically.

Non-Functional Requirements

The system is designed to satisfy non-functional requirements related to performance, scalability, security, reliability, maintainability, and operational efficiency.

To ensure high performance, the platform targets API response times below 300ms and optimized page load speeds through Azure Static Web Apps and lightweight frontend architecture. Backend processing through Azure Functions enables rapid request handling with reduced infrastructure overhead.

The platform is designed to support multiple concurrent users through Azure's scalable serverless infrastructure, allowing backend resources to scale dynamically based on user demand and API workload.

Availability and reliability are ensured through Microsoft Azure's managed cloud infrastructure, which provides high uptime, secure hosting, and reliable cloud service integration for frontend, backend, database, and storage services.

Security is implemented using Microsoft Authentication, secure role-based access control, protected API endpoints, and database-level validation mechanisms. Uploaded tutor certificates and sensitive user information are securely managed through Azure Blob Storage and Azure SQL Database.

Data durability and cloud-based backup support are achieved through managed Azure storage services, ensuring secure persistence of user records, tutor information, uploaded certificates, and booking history.

The system architecture follows a modular cloud-native approach that improves maintainability and simplifies future feature expansion. Automated deployment pipelines using GitHub Actions further improve operational efficiency and reduce deployment complexity.

Finally, the project is designed to remain cost-efficient during academic development and testing phases by leveraging Azure student resources and serverless cloud services while still maintaining enterprise-grade scalability and security capabilities.

2.2 PROPOSED SOLUTION OVERVIEWS

The proposed solution is a cloud-based tutor discovery and booking platform designed using a modular and service-oriented architecture to ensure scalability, maintainability, and efficient cloud resource utilization. The system enables students to browse tutor profiles, verify tutor information, book academic sessions, and interact with AI-assisted services through a centralized web application.

At the presentation layer, the frontend application developed using HTML, CSS, and JavaScript is hosted

on Azure Static Web Apps. The frontend communicates securely with backend API endpoints implemented using Azure Functions. These serverless APIs manage all core platform functionalities, including authentication, tutor profile retrieval, booking management, chatbot interactions, and certificate verification workflows.

The backend orchestrates business logic and handles requests related to students, tutors, administrators, booking operations, and AI-based approval processes. Persistent storage of user data, tutor details, schedules, ratings, booking records, and verification results is managed through Azure SQL Database. Tutor-uploaded certificates and related documents are securely stored in Azure Blob Storage.

Artificial intelligence services are integrated into the platform using Azure OpenAI models. Students can interact with an AI-powered chatbot for assistance and navigation support. Additionally, when tutors upload qualification certificates, the system performs AI-assisted fake document detection and authenticity analysis. Based on generated confidence scores, tutor applications are automatically approved, rejected, or forwarded for manual administrative review.

The application also integrates Microsoft Authentication through Azure App Registration to provide secure login and role-based access management. GitHub Actions automates the CI/CD workflow by enabling seamless frontend deployment and backend update processes directly from the source code repository.

This architectural approach combines serverless computing, managed database services, cloud storage, AI-powered automation, and deployment automation to deliver a secure, scalable, and intelligent tutor discovery platform.

2.3 CLOUD DEPLOYMENT STATERGY

The deployment strategy leverages Microsoft Azure as the primary cloud platform, utilizing a combination of Platform-as-a-Service (PaaS) and serverless cloud services to reduce infrastructure management complexity while ensuring scalability, reliability, and efficient resource utilization. The application is designed as a cloud-native web platform where frontend, backend, authentication, database, storage, and AI services are integrated within the Azure ecosystem.

The frontend application developed using HTML, CSS, and JavaScript is deployed through Azure Static Web Apps, providing secure hosting, automatic HTTPS support, global content delivery, and seamless integration with GitHub Actions for automated CI/CD deployment workflows. The backend layer is implemented using Azure Functions, where serverless API endpoints handle tutor profile management, booking workflows, authentication handling, chatbot interactions, and AI-assisted verification processes. This serverless deployment strategy ensures that compute resources are consumed only when API requests are triggered, improving scalability and cost efficiency.

Azure SQL Database serves as the centralized relational database for storing user information, tutor profiles, booking records, ratings, schedules, and verification data. Uploaded tutor certificates and related documents are stored securely in Azure Blob Storage, enabling scalable and durable cloud-based file management.

The system integrates Microsoft Authentication using Azure App Registration to provide secure login and role-based access control for students, tutors, and administrators. Azure OpenAI services are deployed as part of the intelligent automation layer, supporting AI-powered chatbot functionality and tutor certificate verification workflows. When tutors upload certificates, the backend invokes Azure OpenAI models to analyze uploaded documents for fake certificate detection and authenticity validation. Based on generated confidence scores, the system automatically routes applications for approval, rejection, or manual administrative review.

The overall workflow begins with users interacting through the frontend application hosted on Azure Static Web Apps. API requests are securely routed to Azure Functions, which process business logic and communicate with Azure SQL Database and Azure Blob Storage. AI-related operations are executed through Azure OpenAI integrations, ensuring intelligent automation and improved platform efficiency. This layered deployment architecture enables secure communication, modular scalability, simplified maintenance, and independent service expansion within the cloud environment.

2.4 INFRASTRUCTURE REQUIREMENTS

Compute Resources:

• Azure Functions: Consumption Plan used for hosting serverless backend APIs responsible for authentication, tutor management, booking workflows, chatbot processing, and AI-based verification operations.

• Azure Static Web Apps: Used for hosting the frontend application with integrated CDN support, automatic SSL configuration, and GitHub-based CI/CD deployment workflows.

• Azure OpenAI Services: Cloud-based AI processing resources utilized for chatbot interaction, tutor certificate analysis, and fake document detection workflows.

Storage Requirements:

• Azure SQL Database: Relational database service used for storing tutor profiles, student records, booking details, ratings, schedules, authentication metadata, and verification results with automated cloud-managed backup support.

• Azure Blob Storage: Hot tier blob storage used for secure storage of uploaded tutor certificates, profile images, and verification-related documents with scalable cloud-based file management capabilities.

2.5 AZURE SERVICES MAPPING AND JUSTIFICATION

Table 6: Service Mapping

Requirement	Azure Service	SKU/Tier	Purpose	Justification	Est. Monthly Cost (₹)
Frontend Hosting	Azure Static Web Apps	Standard	Host responsive web application	Integrated CI/CD, SSL	Free (Student Credit)
Backend APIs	Azure Functions	Consumption Plan	Handle backend logic and API endpoints	Serverless, scalable, cost-efficient	500
Database Storage	Azure SQL Database	Basic Tier	Store tutor, student, and booking data	Reliable relational cloud database	Free (Student Credit)
File Storage	Azure Blob Storage	Hot Tier	Store tutor certificates and profile images	Scalable and durable cloud storage	300
Authentication	Azure App Registration	Standard	Microsoft-based secure login	Secure role-based authentication	Free

AI Chatbot	Azure OpenAI Services	Pay-per-use	Student assistance and chatbot support	Intelligent conversational interaction	300
AI Verification	Azure OpenAI Vision Models	Pay-per-use	Fake certificate detection and validation	Automated tutor approval workflow	400
Monitoring & Logs	Azure Monitor	Standard	Monitor application performance and logs	Improve reliability and issue tracking	Free
CI/CD Automation	GitHub Actions	Basic	Automated frontend and backend deployment	Seamless GitHub integration	Free

CHAPTER 3 - DEVOPS IMPLEMENTATION

3.1 CONTINUOUS INTEGRATION AND DEPLOYMENT STATERGY

The project uses a CI/CD pipeline implemented with GitHub Actions to automate frontend deployment, backend integration updates, and cloud deployment workflows. The deployment strategy combines serverless Azure services, source control integration, and partial Infrastructure-as-Code (IaC) practices to simplify deployment management and improve development efficiency.

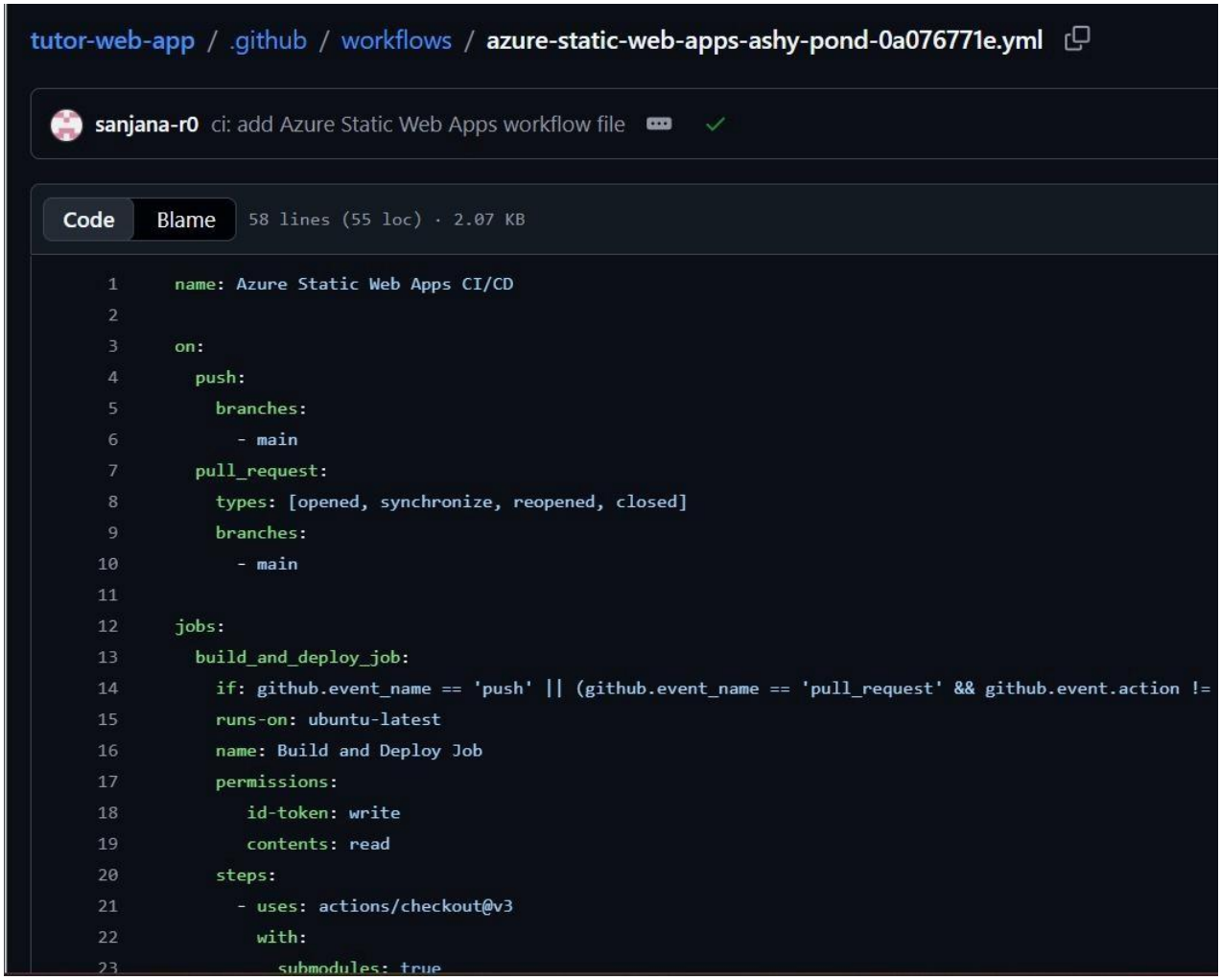


Figure 1. GitHub Actions Workflow Configuration

The deployment workflow primarily consists of two major stages:

Frontend Deployment:

Builds and deploys the frontend web application to Azure Static Web Apps. **Backend Integration and**

API Connectivity:

Integrates frontend services with Azure Function App APIs used for authentication, tutor management, booking workflows, chatbot processing, and AI-assisted tutor verification.

The frontend application developed using HTML, CSS, and JavaScript is automatically compiled and deployed through GitHub Actions workflows. Azure Static Web Apps handles hosting, SSL configuration, and content delivery, while GitHub Actions automates the build and deployment process.

Backend API services are implemented using Azure Functions. API endpoints manage tutor profile handling, booking operations, chatbot interactions, authentication workflows, and AI-powered certificate verification processes. Environment configurations such as API URLs, authentication parameters, and Cross-Origin Resource Sharing (CORS) settings are maintained during deployment.

Secure access to Azure resources is managed using Azure credentials and secrets stored securely within GitHub Secrets. These secrets are used for authenticating deployment workflows and enabling secure

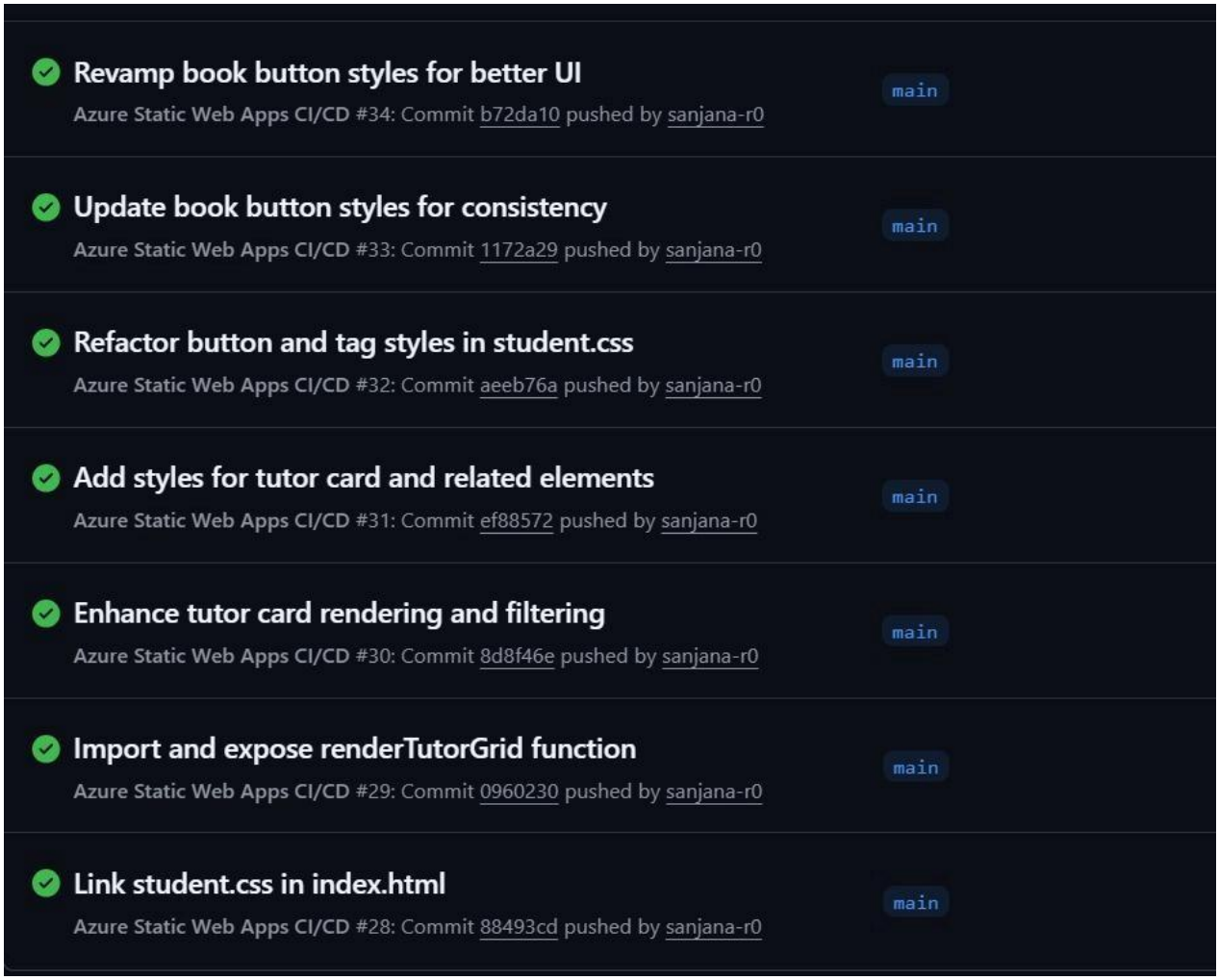


Figure 2. Successful GitHub Actions Deployment Runs

The deployment process also establishes secure connectivity between Azure Functions, Azure SQL Database, Azure Blob Storage, and Azure OpenAI services. This integration enables centralized cloud-based data handling, certificate storage, AI-powered verification, and chatbot functionalities within the platform.

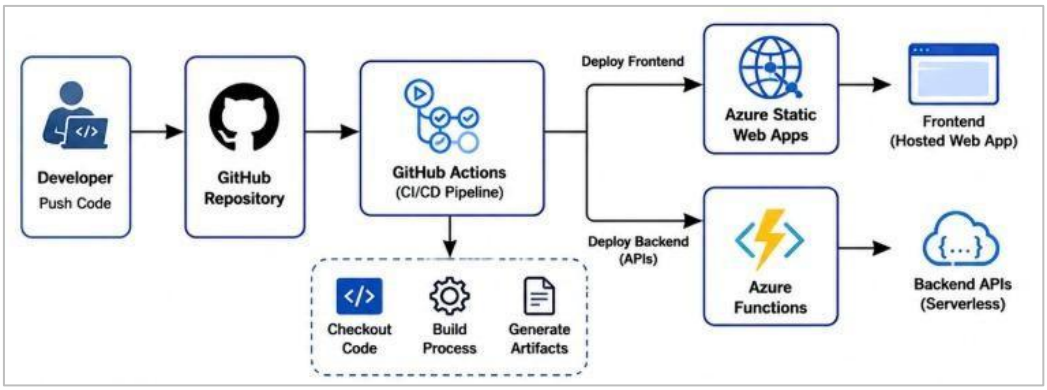


Figure 3. CI/CD Deployment Workflow

The automated deployment workflow significantly reduces manual deployment effort, improves consistency across updates, and ensures rapid synchronization between source code changes and cloud-hosted application services.

Required GitHub Secrets

Table 7. List of Github Secrets

S.No	GitHub Secret
1	AZURE_CLIENT_ID
2	AZURE_CLIENT_SECRET
3	AZURE_STATIC_WEB_APPS_API_TOKEN
4	AZURE_STORAGE_ACCOUNT_NAME
5	AZURE_SUBSCRIPTION_ID
6	AZURE_TENANT_ID
7	AZURE_OPENAI_KEY
8	DB_CONNECTION_STRING
9	STORAGE_ACCOUNT_KEY
10	MICROSOFT_AUTH_CLIENT_ID
11	MICROSOFT_AUTH_CLIENT_SECRET

3.2 TERRAFORM INFRASTRUCTURE-AS-CODE (IaC)

Terraform Infrastructure-as-Code (IaC)

Terraform was partially implemented in the project to demonstrate Infrastructure-as-Code (IaC) concepts for automating Azure resource provisioning and cloud infrastructure management. The implementation focused on provisioning selected Azure resources such as Resource Groups, Storage Accounts, and App Service configurations within the Azure environment.

Terraform File Structure

The Terraform implementation follows a standard three-file structure for organizing infrastructure configurations:

- **main.tf** – Contains the primary Azure resource definitions and infrastructure configurations.
- **variables.tf** – Stores configurable input values such as resource names, Azure region, and deployment settings.
- **outputs.tf** – Defines deployment outputs such as generated resource names and storage account details for verification and reuse.

Core Infrastructure Components

The Infrastructure-as-Code configuration provisions selected Azure resources and organizes them within dedicated Azure Resource Groups for centralized cloud resource management.

Azure Resource Group:

Terraform provisions Azure Resource Groups that logically organize cloud services and deployment resources within the Azure environment.

Azure Storage Account:

Terraform provisions Azure Storage Accounts used for Blob Storage integration and secure cloud-based file management.

Azure Service Plan:

Terraform configurations also demonstrate provisioning of Azure App Service Plans required for hosting and managing Azure cloud services.

Storage Integration

Blob Storage Support:

The provisioned Storage Account supports Blob Storage integration for storing tutor certificates, profile images, and verification-related documents within the cloud infrastructure.

Serverless API Integration

Azure Function App Support:

The Terraform-managed resources support integration with Azure Functions used for backend APIs, tutor management workflows, authentication handling, chatbot services, and AI-powered certificate verification processes.

CI/CD Integration

The Terraform setup is designed to integrate with GitHub Actions deployment workflows. During CI/CD execution, Azure credentials and deployment secrets are securely passed through GitHub Secrets. Terraform configurations provision Azure resources using automated deployment commands. Generated outputs from Terraform configurations are used during deployment validation and cloud integration workflows.

main.tf terraform.tfstate

infra > main.tf

```
provider "azurerm" {
  features {}
}

resource "azurerm_resource_group" "rg" {
  name     = "tutor-app-rg-1"
  location = "Central India"
}

resource "azurerm_storage_account" "storage" {
  name                     = "tutorstorage98765"
  resource_group_name      = azurerm_resource_group.rg.name
  location                 = azurerm_resource_group.rg.location
  account_tier             = "Standard"
  account_replication_type = "LRS"
}

resource "azurerm_service_plan" "plan" {
  name                = "tutor-app-plan"
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
  os_type             = "Linux"
  sku_name            = "Y1"
}

resource "azurerm_linux_function_app" "function" {
  name                       = "tutor-function-app-12345"
  location                   = azurerm_resource_group.rg.location
  resource_group_name        = azurerm_resource_group.rg.name
  service_plan_id            = azurerm_service_plan.plan.id
  storage_account_name       = azurerm_storage_account.storage.name
  storage_account_access_key = azurerm_storage_account.storage.primary_access_key

  site_config {}
}
```

Figure 3. Terraform Azure Resource Configuration

3.3 CONTAINERIZATION STRATERGY (DOCKER)

Docker concepts were studied and demonstrated as part of the DevOps implementation process to understand containerization workflows and cloud-native deployment strategies. The implementation focused on understanding how applications can be packaged into lightweight and portable containers for scalable cloud deployment.

Docker File Structure

The Docker configuration demonstrates a layered image-building approach where application dependencies, runtime configurations, and execution environments are packaged into isolated containers.

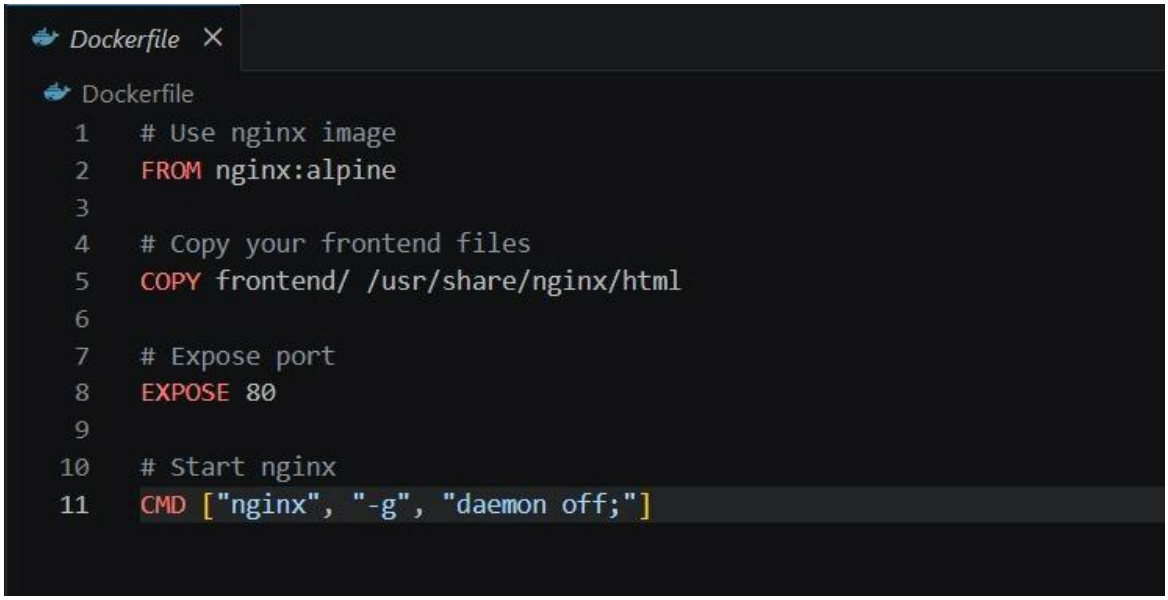


Figure 4. Docker file

Containerization Concepts:

Application Packaging:

Docker containers package application code, dependencies, runtime libraries, and configurations into a single portable environment.

Environment Consistency:

Containerization ensures consistent execution across development, testing, and deployment environments without dependency conflicts.

Scalability and Deployment Support:

Docker-based deployment strategies simplify cloud-native application hosting and improve deployment portability across infrastructure environments.

CI/CD Integration:

Docker concepts were integrated with GitHub Actions workflows to understand automated build and deployment strategies within DevOps pipelines.

Although the final deployed implementation primarily uses Azure Functions and Azure Static Web Apps, Docker-based containerization concepts were explored as part of the overall DevOps and cloud deployment learning process.

3.4 KUBERNETES ORCHESTRATION

Azure Kubernetes Service (AKS) was implemented in the project to understand container orchestration, scalability management, and cloud-native deployment practices within the Microsoft Azure ecosystem. AKS provides centralized management for containerized workloads while simplifying cluster administration, scaling, networking, and deployment operations.

1. **Cluster and Deployment Design**

Cluster Structure:

The AKS cluster was configured within the Azure environment to study Kubernetes-based workload orchestration and cloud-native deployment architecture. Azure CNI networking and managed Kubernetes services were utilized for cluster communication and resource management.

Workload Management:

Containerized application workloads were deployed and managed within the AKS environment using Kubernetes deployment configurations. Kubernetes concepts such as pods, deployments, namespaces, and service exposure were explored as part of the implementation.

Ingress and Traffic Routing:

Ingress and service routing concepts were implemented to understand how external traffic can be securely routed to containerized applications running within Kubernetes clusters.

Configuration and Secrets Management:

Kubernetes configuration management practices such as environment variables, deployment configuration files, and secure secret handling mechanisms were studied within the AKS environment.

2. Scaling, Resilience, and Security

Pod Scaling:

AKS scaling concepts were implemented to understand how Kubernetes dynamically manages workloads and adjusts container instances based on application demand and resource utilization.

Node Management:

Cluster resource allocation and node management concepts were explored to understand scalable cloud infrastructure management within Kubernetes environments.

Resilience and Availability:

Kubernetes deployment strategies such as rolling updates, workload monitoring, and fault tolerance mechanisms were studied to improve application reliability and deployment stability.

The AKS implementation provided practical exposure to Kubernetes orchestration, scalable cloud-native deployment strategies, and enterprise-level container management within Microsoft Azure cloud infrastructure.

3.5 GENAI INTEGRATION AND AZURE AI SERVICE MAPPING

1. AI-Powered Chatbot and Tutor Verification (Azure OpenAI Services)

The project integrates Azure OpenAI Services to provide intelligent automation features such as student chatbot assistance and AI-based tutor certificate verification. These AI capabilities improve user interaction, automate administrative workflows, and enhance platform reliability.

Implementation

The Azure OpenAI integration is connected with Azure Functions, allowing AI-related operations to execute independently from the frontend application. This serverless integration enables scalable and efficient handling of AI requests while maintaining optimized cloud resource utilization.

Process

1. Students interact with an AI-powered chatbot integrated within the platform for assistance, navigation support, and academic-related queries.
2. Tutors upload qualification certificates and verification documents through the platform interface.
3. Uploaded certificate images are securely stored in Azure Blob Storage.
4. Azure OpenAI vision capabilities analyze the uploaded documents for authenticity validation and fake

certificate detection.

5. The AI system generates confidence scores and verification results based on image analysis and document consistency checks.

2. Automated Verification and Moderation Workflow

The AI-powered verification workflow ensures that tutor registrations comply with platform trust and security requirements before approval.

Implementation

Azure OpenAI Services are integrated with backend APIs to automate tutor approval workflows and reduce manual administrative effort during tutor onboarding processes.

Process

Every uploaded tutor certificate is analyzed through Azure OpenAI models to detect inconsistencies, suspicious patterns, or potential fake credentials.

Decision Flow

The AI verification system generates automated outcomes based on confidence scores:

- High-confidence authentic certificates are automatically approved.
- Suspicious or inconsistent documents are automatically rejected.
- Medium-confidence cases are forwarded to administrators for manual review.

This intelligent verification workflow improves platform trust, reduces manual verification effort, and enhances the overall reliability and security of the tutor onboarding process.

CHAPTER 4 - CLOUD OPERATIONS AND SECURITY

1.1 DEVSECOPS INTEGRATION

Security is integrated throughout the development and deployment lifecycle using DevSecOps practices embedded directly within the GitHub Actions CI/CD workflow. Automated security analysis is performed during every code commit and deployment process to identify vulnerabilities, insecure coding patterns, and potential application security risks before deployment to production environments.

The project integrates **GitHub CodeQL** and **OWASP ZAP** within the CI/CD pipeline to automate security testing and vulnerability assessment.

CodeQL Static Security Analysis

CodeQL is integrated using a dedicated codeql.yml workflow within GitHub Actions. The CodeQL workflow automatically scans the application source code whenever changes are pushed to the repository.

The analysis checks for:

- Insecure coding practices
- Vulnerable code patterns
- Improper environment variable handling
- Potential injection vulnerabilities
- Authentication and authorization issues
- Security misconfigurations

The automated scanning process generates security warnings and vulnerability reports directly within the GitHub Actions workflow execution logs, enabling early identification and remediation of security issues during development.

OWASP ZAP Security Testing

OWASP ZAP is integrated using a dedicated zap.yml workflow within the CI/CD pipeline for automated dynamic application security testing.

The ZAP workflow performs:

- Automated vulnerability scanning
- Detection of insecure endpoints
- Security header validation
- Common web application attack detection

- Runtime security assessment

The scans generate warnings, alerts, and vulnerability reports during deployment execution, helping identify security weaknesses within the web application and API endpoints.

Secure CI/CD Workflow Integration

The DevSecOps workflows are automatically triggered during every commit and deployment cycle through GitHub Actions. This ensures continuous security monitoring and automated vulnerability detection throughout the software development lifecycle.

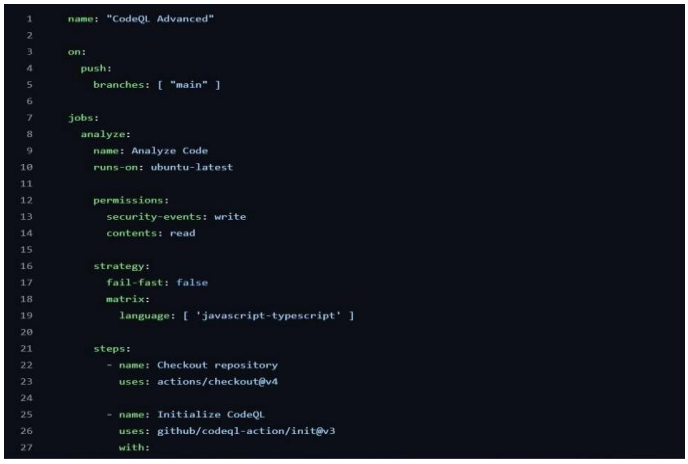
Sensitive credentials and deployment configurations are securely managed using GitHub Secrets and Azure environment configurations. Security-related findings generated by CodeQL and OWASP ZAP are reviewed during development to improve application security and reduce deployment risks.

Demo Security Findings

Example security findings detected during automated scanning include:

- Security warning: use of environment variables outside secure scope
- Potential insecure configuration handling
- Vulnerability alerts identified through OWASP ZAP scans
- Runtime endpoint security warnings

Figure 5. Github Workflow Showing Lint Errors



```yaml
name: "CodeQL Advanced"

on:
  push:
    branches: [ "main" ]

jobs:
  analyze:
    name: Analyze Code
    runs-on: ubuntu-latest

    permissions:
      security-events: write
      contents: read

    strategy:
      fail-fast: false
      matrix:
        language: [ 'javascript-typescript' ]

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Initialize CodeQL
        uses: github/codeql-action/init@v3
        with:
```

Figure 6. CodeQL Security Workflow

```yaml
name: OWASP ZAP Security Scan

on:
  push:
    branches: [ "main" ]

jobs:
  zap_scan:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Run ZAP Scan
        run: |
          docker run --rm \
            ghcr.io/zaproxy/zaproxy:stable \
            zap.sh -cmd \
            -quickurl https://ashy-pond-0a076771e.2.azurestaticapps.net \
            -quickprogress
```

Figure 7. OWASP ZAP Security Workflow

```
PASS: SOAP XML Injection [90029]
PASS: WSDL File Detection [90030]
PASS: Loosely Scoped Cookie [90033]
PASS: Cloud Metadata Potentially Exposed [90034]
PASS: Server Side Template Injection [90035]
PASS: Server Side Template Injection (Blind) [90036]
PASS: Remote OS Command Injection (Time Based) [90037]
PASS: NoSQL Injection - MongoDB (Time Based) [90039]
WARN-NEW: Cross-Domain JavaScript Source File Inclusion [10017] x 8
        https://ashy-pond-0a076771e.2.azurestaticapps.net/ (200 OK)
        https://ashy-pond-0a076771e.2.azurestaticapps.net/robots.txt (404 Not Found)
        https://ashy-pond-0a076771e.2.azurestaticapps.net/robots.txt (404 Not Found)
        https://ashy-pond-0a076771e.2.azurestaticapps.net/robots.txt (404 Not Found)
        https://ashy-pond-0a076771e.2.azurestaticapps.net (200 OK)
WARN-NEW: Missing Anti-clickjacking Header [10020] x 2
        https://ashy-pond-0a076771e.2.azurestaticapps.net/ (200 OK)
        https://ashy-pond-0a076771e.2.azurestaticapps.net (200 OK)
WARN-NEW: Strict-Transport-Security Header Not Set [10035] x 2
        https://ashy-pond-0a076771e.2.azurestaticapps.net/robots.txt (404 Not Found)
        https://ashy-pond-0a076771e.2.azurestaticapps.net/sitemap.xml (404 Not Found)
WARN-NEW: Content Security Policy (CSP) Header Not Set [10038] x 4
        https://ashy-pond-0a076771e.2.azurestaticapps.net/ (200 OK)
        https://ashy-pond-0a076771e.2.azurestaticapps.net/robots.txt (404 Not Found)
        https://ashy-pond-0a076771e.2.azurestaticapps.net (200 OK)
        https://ashy-pond-0a076771e.2.azurestaticapps.net/sitemap.xml (404 Not Found)
WARN-NEW: Permissions Policy Header Not Set [10063] x 5
```

Figure 8. Security Warnings Generated in GitHub Actions Workflow

These DevSecOps integrations improve security visibility, automate vulnerability detection, and strengthen the overall reliability and security posture of the Azure Tutor Platform.

1.2 MONITORING AND OBSERVABILITY

Comprehensive monitoring and observability are implemented using Azure Application Insights, Azure Monitor, and Grafana dashboards to provide real-time visibility into application performance, system health, server activity, and operational metrics across the Azure Tutor Platform.

Azure Application Insights is integrated with the application to continuously collect telemetry data related to API performance, server response time, request failures, exceptions, CPU utilization, memory consumption, and browser-side errors. The monitoring setup enables centralized tracking of both frontend and backend application behavior within the Azure environment.

The Application Insights dashboard provides detailed operational metrics including:

- Failed request counts
- Server response time
- Average page load time
- Server exceptions
- CPU utilization
- Memory availability
- Browser exceptions
- Process I/O statistics

These metrics help identify application bottlenecks, runtime failures, performance degradation, and unusual system behavior during application execution.

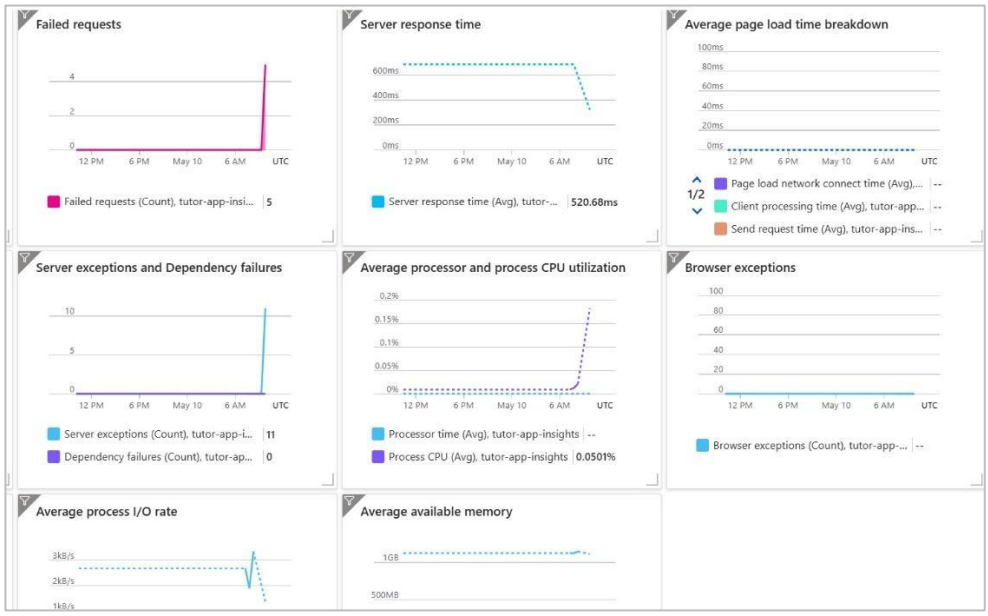


Figure 9. Azure Application Insights Monitoring Dashboard

Grafana is additionally integrated for visualizing real-time monitoring data and server request metrics. Grafana dashboards provide graphical insights into request traffic, system activity, and operational trends, enabling simplified monitoring and performance analysis.

The Grafana dashboards support:

- Real-time server request visualization
- Request traffic monitoring
- Time-series metric analysis
- Performance trend observation
- Centralized operational dashboards

Azure Monitor and Application Insights telemetry collectively support proactive monitoring and operational analysis of the deployed cloud services. These monitoring integrations improve observability, simplify troubleshooting, and assist in identifying application performance issues during deployment and runtime operations.

Figure 10. Grafana Dashboard

1.3 ACCESS CONTROL

Access control is implemented at both infrastructure and application levels using secure authentication mechanisms and role-based authorization principles. Microsoft Authentication is integrated using Azure App Registration to provide secure login functionality for students, tutors, and administrators.

At the application level, authenticated users are granted controlled access based on their assigned roles:

- Students can browse tutor profiles and manage bookings.
- Tutors can manage profile information and uploaded certificates.
- Administrators can review tutor verification requests and platform activities.

Backend API endpoints implemented using Azure Functions validate authentication tokens and enforce secure access to protected resources and operations. Sensitive operations such as tutor approval workflows and administrative functionalities are restricted to authorized users only.

Secure credential management is maintained through GitHub Secrets and Azure environment configurations. Sensitive information such as API keys, database connection strings, authentication credentials, and storage account configurations are not hardcoded within the application source code.

Database access is securely managed through Azure SQL Database authentication mechanisms, while uploaded tutor certificates and verification documents stored in Azure Blob Storage are protected through controlled cloud access policies.

These access control mechanisms improve platform security, restrict unauthorized access, and ensure secure communication between frontend applications, backend APIs, cloud databases, and AI-powered services within the Azure environment.

CHAPTER 5 - RESULTS AND DISCUSSION

5.1 IMPLEMENTATION SUMMARY

The Tutor-Match Platform was successfully implemented and deployed on Microsoft Azure, achieving the primary objectives defined in the project scope. The system provides a secure and scalable tutor discovery platform where students can browse tutor profiles, manage bookings, interact with AI-powered chatbot services, and access authenticated cloud-based functionalities.

The frontend application was successfully deployed using Azure Static Web Apps with integrated GitHub Actions CI/CD workflows for automated deployment and version management. Backend APIs were implemented using Azure Functions to manage authentication, tutor profile handling, booking workflows, AI verification operations, and database communication. Azure SQL Database was integrated for structured storage of tutor profiles, student information, booking records, and verification details, while Azure Blob Storage handled secure storage of uploaded tutor certificates and related documents.

The integration of Microsoft Authentication through Azure App Registration enabled secure login and role-based access management for students, tutors, and administrators. AI-powered functionalities implemented using Azure OpenAI services successfully provided chatbot assistance and automated tutor certificate verification workflows. Uploaded tutor certificates were analyzed for authenticity validation and fake document detection, with applications automatically routed for approval, rejection, or manual administrative review based on generated confidence scores.

The CI/CD pipeline implemented using GitHub Actions successfully automated frontend deployment, backend updates, security scanning, and deployment workflows. Integrated DevSecOps workflows using CodeQL and OWASP ZAP continuously scanned the application during every commit and deployment cycle, helping identify vulnerabilities and security warnings during development.

Monitoring and observability were implemented using Azure Application Insights and Grafana dashboards, providing real-time visibility into API performance, server response times, request failures, CPU utilization, memory consumption, and operational metrics. The deployed system demonstrated stable cloud integration, scalable architecture, automated deployment workflows, and secure cloud-based service management within the Azure ecosystem.

5.2 CHALLENGES FACED AND RESOLUTIONS

One of the primary challenges faced during development was integrating multiple Azure cloud services into a unified and secure workflow. Managing communication between Azure Static Web Apps, Azure Functions, Azure SQL Database, Blob Storage, Azure OpenAI services, and authentication systems required careful API configuration and secure cloud integration practices.

Another challenge involved implementing AI-assisted tutor certificate verification workflows. Since uploaded certificates required authenticity analysis and fake document detection, integrating Azure OpenAI vision capabilities with backend APIs and Blob Storage required additional validation logic and asynchronous processing workflows. This challenge was resolved by securely storing uploaded documents in Azure Blob Storage and processing verification requests through Azure Functions.

The project also faced challenges in maintaining secure CI/CD deployment pipelines and cloud credentials management. To address this, GitHub Secrets were integrated with GitHub Actions workflows to securely manage API keys, database connection strings, Azure credentials, and deployment configurations without exposing sensitive information within the source code.

During security testing, automated CodeQL and OWASP ZAP workflows generated vulnerability warnings and security alerts during deployment execution. These findings helped identify insecure configurations, environment variable handling issues, and API security concerns early in development. Security configurations and validation logic were improved based on these automated scan results.

Another challenge involved monitoring application behavior and identifying runtime issues within cloud-hosted services. This was addressed by integrating Azure Application Insights and Grafana dashboards to monitor request failures, exceptions, response times, CPU usage, and operational metrics in real time.

5.3 PERFORMANCE OR COST OBSERVATION

Performance Benchmarking

Performance monitoring confirmed the efficiency and stability of the selected Azure cloud services and serverless architecture. Azure Functions successfully handled backend API requests with responsive execution times while supporting scalable serverless processing for authentication, booking workflows, chatbot interactions, and AI-assisted verification.

Azure Application Insights monitoring dashboards provided detailed visibility into server response times, failed requests, browser exceptions, CPU utilization, memory usage, and server-side exceptions. Grafana dashboards additionally visualized real-time server request metrics and operational activity trends for continuous monitoring and observability.

The frontend application hosted through Azure Static Web Apps demonstrated fast page loading

performance and reliable frontend-backend communication through Azure Functions APIs. Azure SQL Database and Blob Storage integrations also provided stable cloud-based data handling and file storage performance during application testing.

Cost Analysis and Optimization

The project was designed to remain highly cost-efficient by utilizing Azure student credits and free-tier cloud services wherever possible. Most Azure resources used in the implementation, including Azure Static Web Apps, Azure Functions, Azure SQL Database basic configurations, GitHub Actions workflows, and monitoring services, were operated within the Azure Student Subscription limits.

The primary cloud resource consuming Azure credits was Azure OpenAI services, which operate using a request-based consumption model. Credits are consumed only when AI chatbot interactions or tutor certificate verification models are actively used within the application. This pay-per-request model helped minimize unnecessary operational cost during development and testing phases.

AKS-related implementations and Kubernetes experimentation also consumed a portion of the available Azure credits during orchestration and infrastructure testing activities.

The serverless architecture significantly reduced infrastructure overhead by ensuring compute resources were consumed only when backend APIs or AI operations were actively triggered. This optimized deployment approach minimized operational expenses while still supporting scalable cloud-native functionality.

5.4 KEY LEARNINGS AND TEAM CONTRIBUTIONS

Throughout the development of the Tutor-Match Platform, the team gained practical experience in cloud-native application development, DevOps practices, AI integration, serverless computing, and secure deployment workflows within Microsoft Azure environments.

One of the major learnings was the implementation of CI/CD automation using GitHub Actions integrated with Azure cloud services. The team gained experience in automating deployment workflows, configuring GitHub Secrets, managing cloud credentials securely, and integrating automated security scanning using CodeQL and OWASP ZAP.

Another important learning involved integrating Azure OpenAI services into real-world application workflows. The implementation of AI-powered chatbot assistance and tutor certificate verification provided practical understanding of intelligent automation, AI model integration, image-based verification workflows, and cloud-based AI processing.

The project also strengthened understanding of Azure cloud services including Azure Static Web Apps,

Azure Functions, Azure SQL Database, Azure Blob Storage, Azure Application Insights, Grafana dashboards, and Azure Kubernetes Service concepts. Working with these services improved the team's knowledge of cloud deployment architecture, serverless backend design, monitoring systems, authentication integration, and cloud resource management.

From a DevOps perspective, the team gained hands-on experience with Infrastructure-as-Code concepts using Terraform, containerization fundamentals using Docker, monitoring and observability practices, and Kubernetes orchestration concepts through AKS experimentation.

In terms of collaboration, the team followed a structured development approach with clear task allocation, regular discussions, GitHub-based version control practices, pull request reviews, and parallel feature development. This collaborative workflow improved project coordination, communication, debugging efficiency, and overall software development productivity throughout the project lifecycle.

CHAPTER 6 - CONCLUSION AND FUTURE WORKS

6.1 CONCLUSION

This project successfully delivered a secure, scalable, and AI-powered tutor discovery platform that helps students connect with reliable and verified tutors through a centralized cloud-based system. The platform enables students to browse tutor profiles, manage bookings, interact with AI-powered chatbot services, and access verified tutor information securely. The integration of Azure OpenAI services for chatbot assistance and AI-based certificate verification improved platform trust, automation, and overall user experience.

The implementation demonstrates practical use of modern cloud-native and DevOps technologies within the Microsoft Azure ecosystem. The frontend was deployed using Azure Static Web Apps, while backend functionalities were implemented using Azure Functions integrated with Azure SQL Database and Azure Blob Storage. GitHub Actions automated the CI/CD workflow, and DevSecOps practices were implemented using CodeQL and OWASP ZAP for automated vulnerability detection and security analysis.

Monitoring and observability were achieved through Azure Application Insights and Grafana dashboards, providing real-time visibility into application performance, request metrics, and operational health. Microsoft Authentication through Azure App Registration ensured secure login and role-based access management for students, tutors, and administrators.

Overall, the project provided valuable hands-on experience in cloud computing, serverless architecture, AI integration, DevOps automation, security practices, monitoring systems, and Kubernetes orchestration concepts while successfully delivering an intelligent and secure Azure-based tutor management platform.

6.2 FUTURE WORKS

The current implementation provides a scalable and extensible foundation for several future enhancements aimed at improving platform intelligence, usability, automation, scalability, and overall student engagement. The architecture and cloud-based deployment model allow the platform to support additional advanced features, improved automation workflows, enhanced communication services, and AI-driven capabilities in future development phases while maintaining flexibility, security, and efficient resource management.

Short-to-Medium Term Enhancements

Advanced Tutor Recommendation System:

Future development can include an AI-powered recommendation engine that suggests tutors based on student learning preferences, subject difficulty, ratings, previous bookings, and academic requirements for more personalized tutor discovery.

Integrated Real-Time Communication:

The platform can be extended with real-time chat and video session integration to improve direct interaction between students and tutors during booking and academic support workflows.

Student Review and Rating System:

A feedback and rating mechanism can be implemented to improve tutor transparency, platform trust, and tutor quality assessment based on student experiences.

Enhanced AI Verification Models:

Future AI workflows can incorporate more advanced document validation techniques and additional verification models to improve tutor authenticity analysis and reduce false-positive verification outcomes.

Long-Term Vision:

Mobile Application Support:

The platform can be expanded into Android and iOS mobile applications to improve accessibility and user convenience across multiple devices.

Payment Gateway Integration:

Secure online payment systems can be integrated to support direct tutor payment processing, session subscriptions, and automated billing workflows.

Scalable Multi-Institution Deployment:

The platform architecture can be expanded to support multiple schools, colleges, and educational institutions through a scalable multi-tenant cloud deployment model.

Advanced Analytics and Insights:

Future versions can integrate analytics dashboards for tracking tutor performance, student engagement trends, booking statistics, and AI-driven academic insights.

These future enhancements will further improve scalability, automation, personalization, and overall platform effectiveness while extending the system into a more comprehensive cloud-based educational support ecosystem.

APPENDICES

Appendix A – Tutor-Match Platform

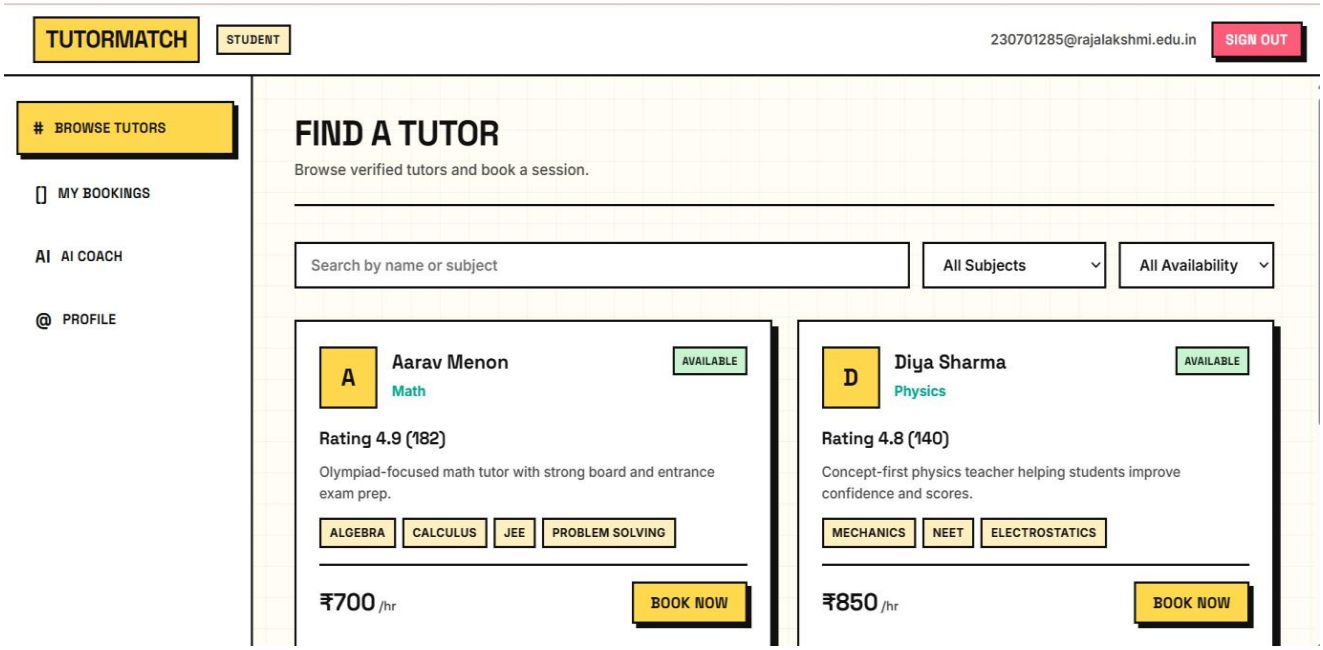


Figure 9. Student Profile

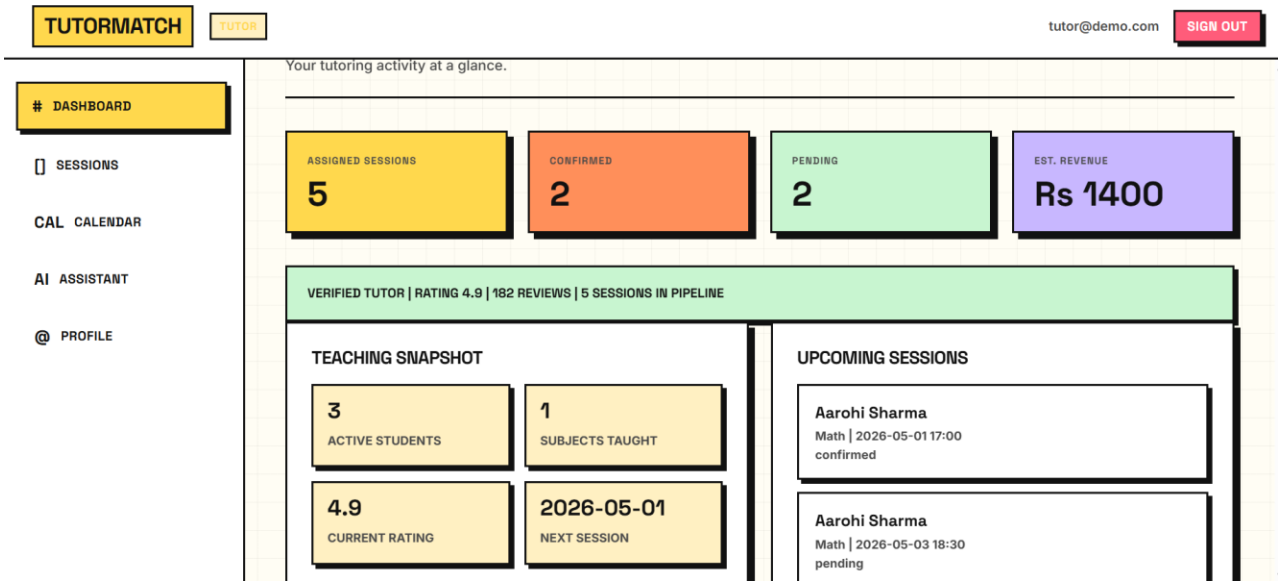


Figure 10. Tutor Profile

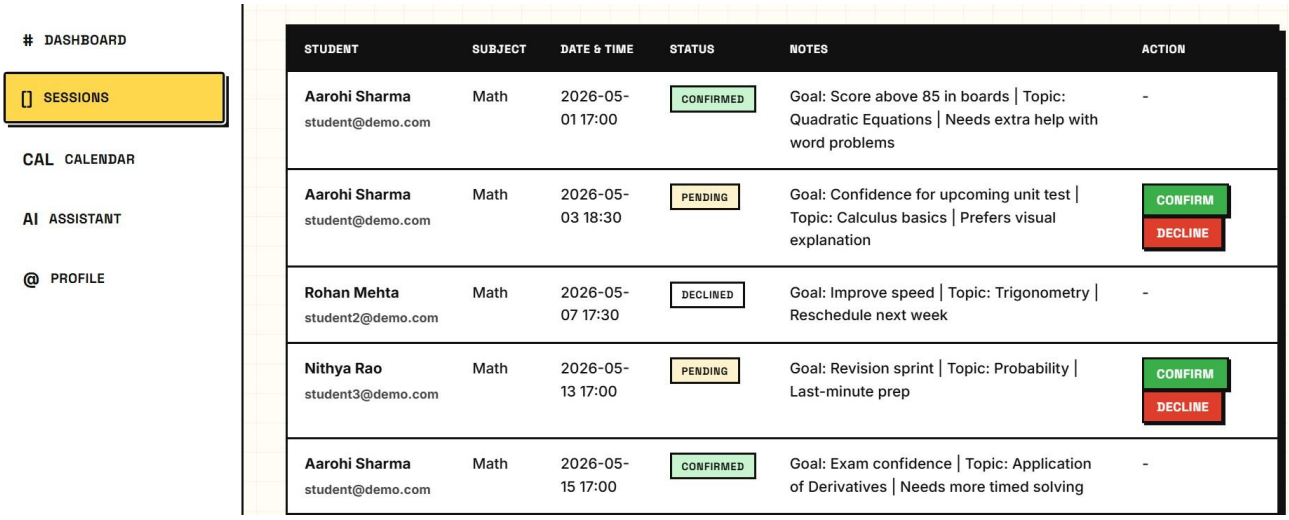


Figure 11. Tutor requests

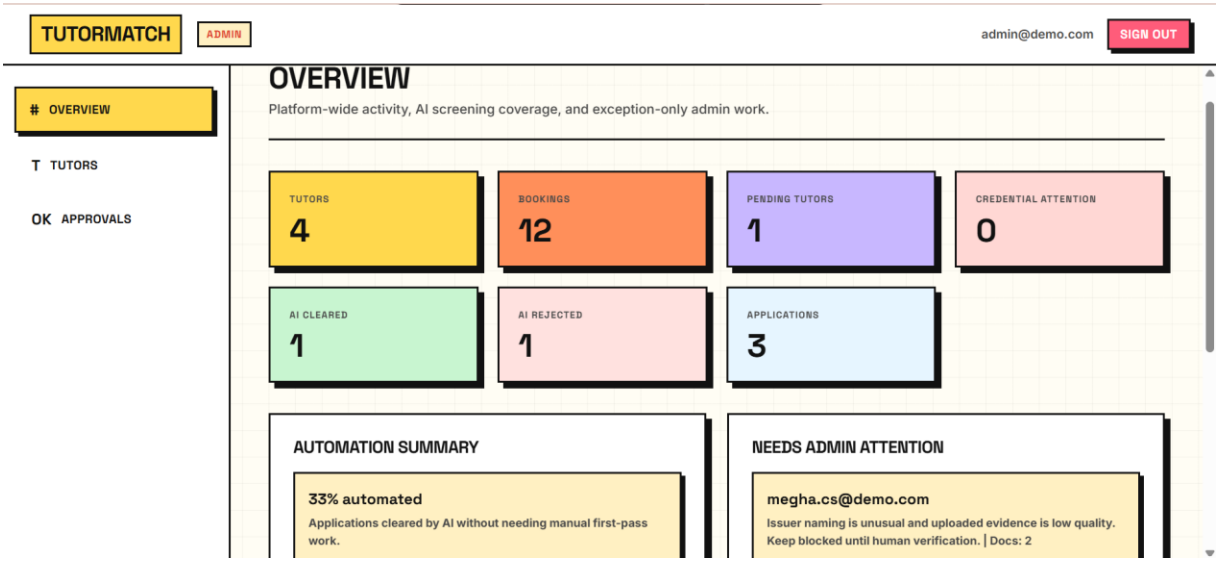


Figure 12. Admin Profile

Appendix B – Pipleline YAMLs and CI/CD

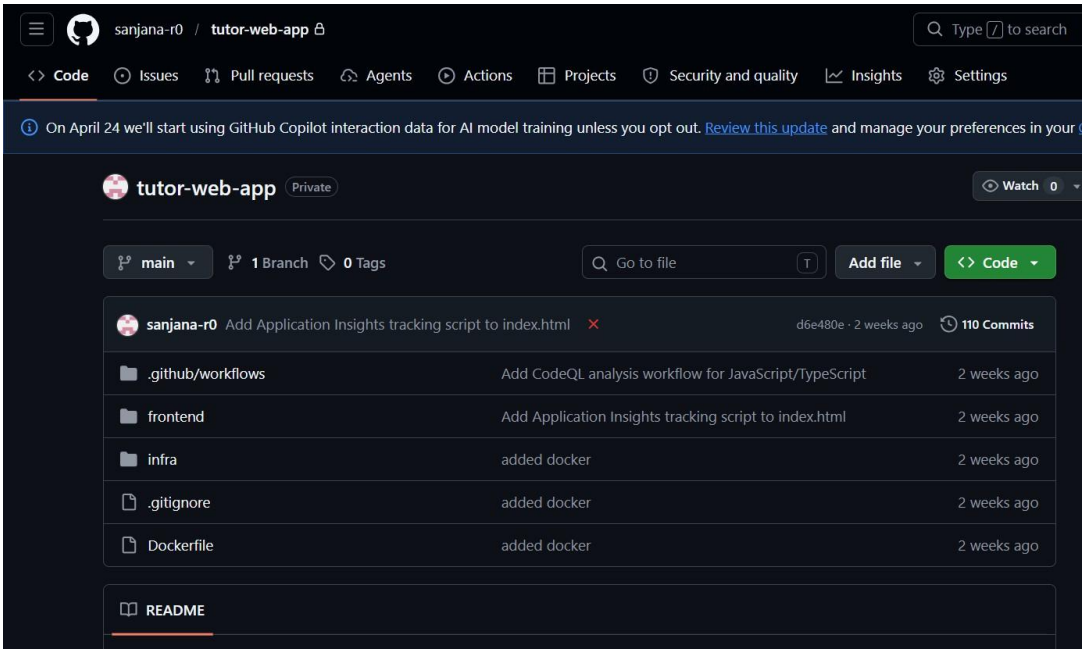


Figure 13. Main branch file

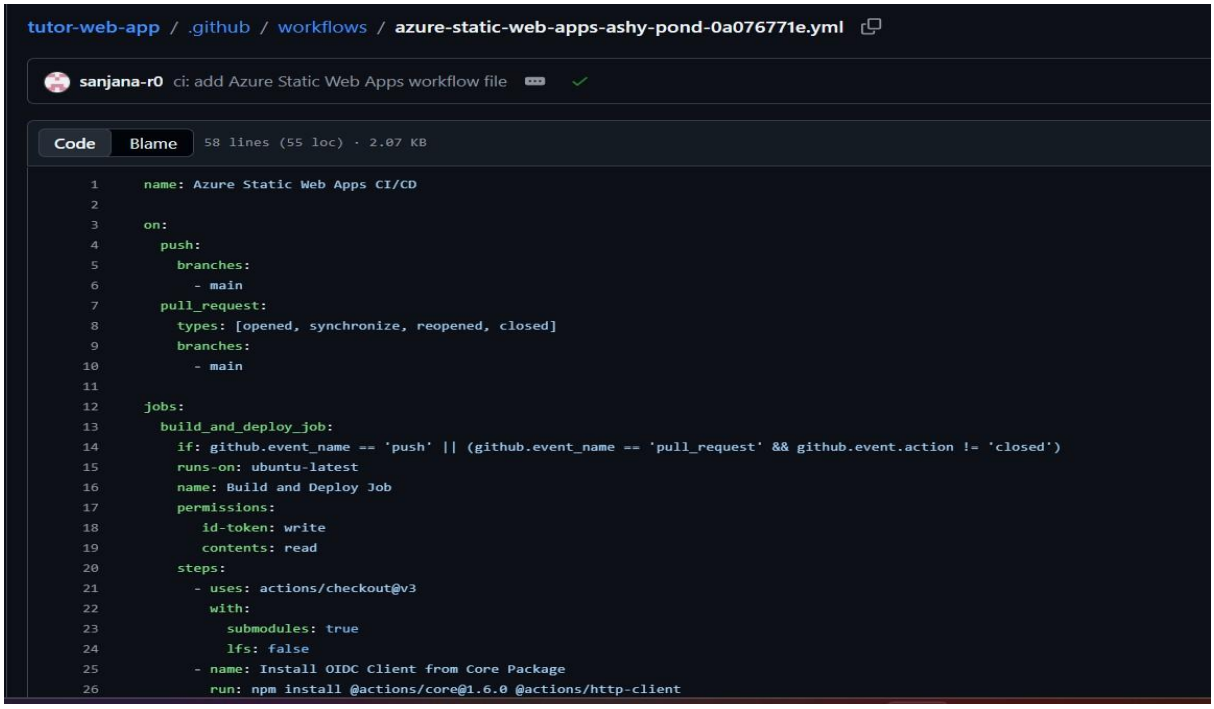


Figure 14. Static-Web App workflow

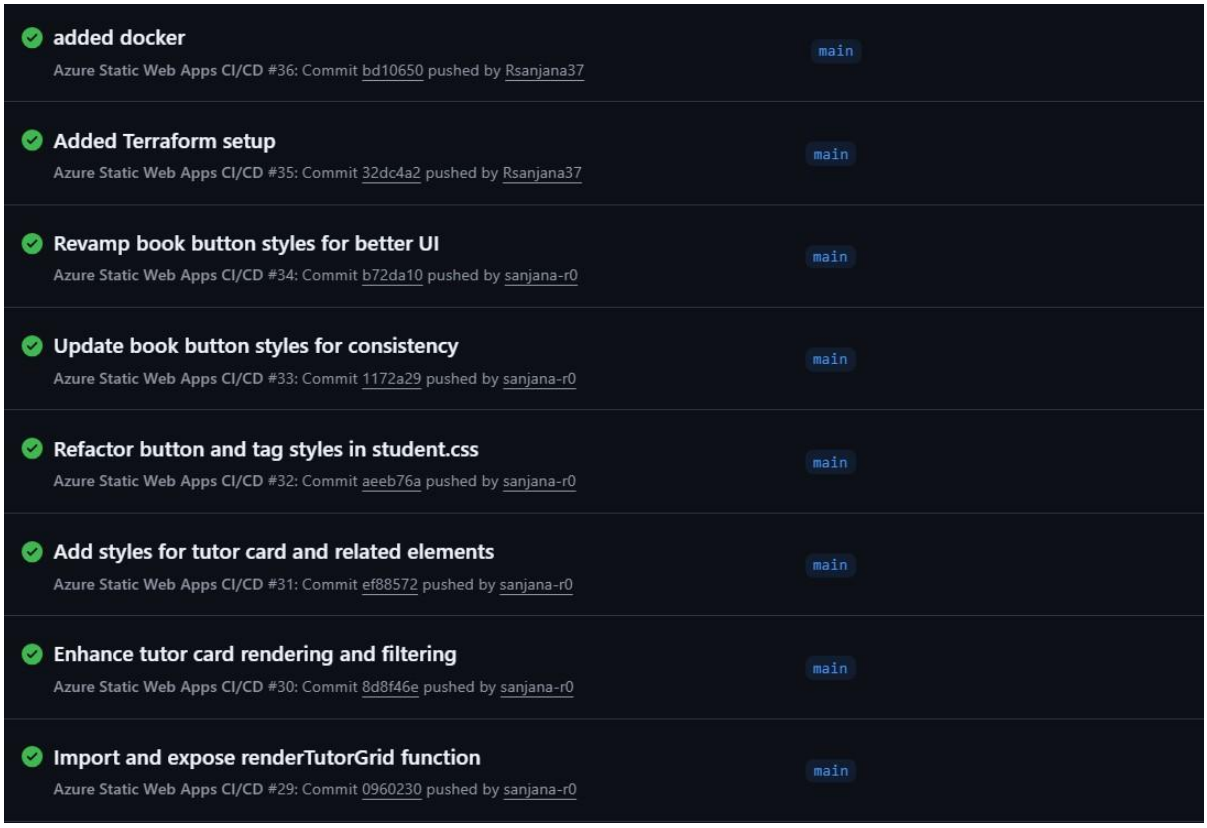


Figure 15. Successful Workflow Runs

Appendix C – Screenshots (AI Integration, Security Scans)

Integrated AI services:

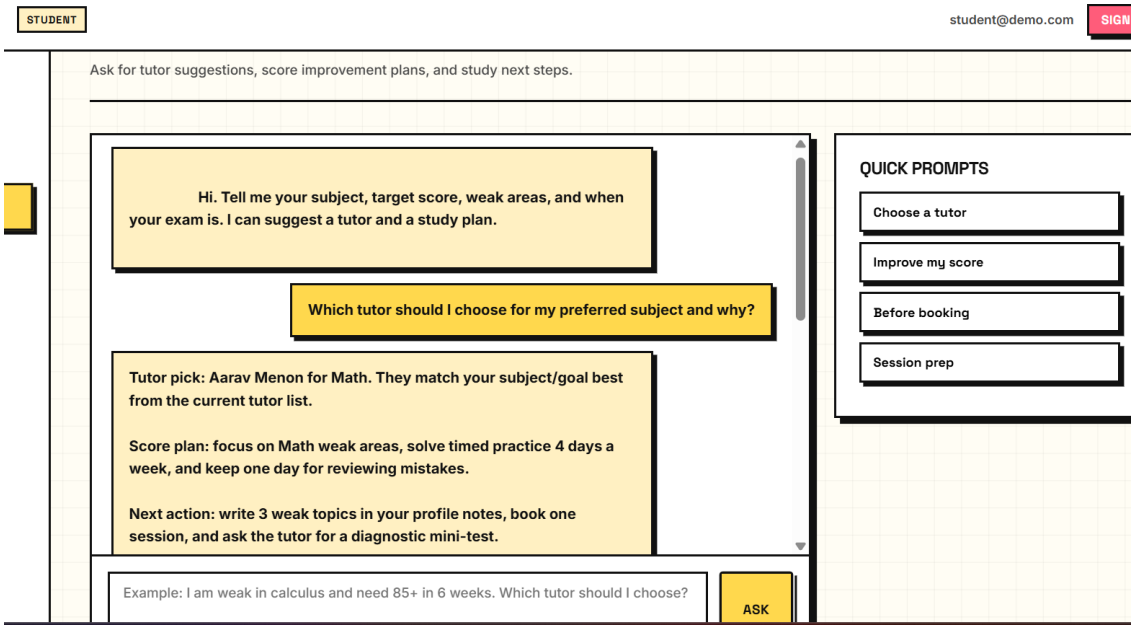


Figure 16. AI Chatbot in Student Account

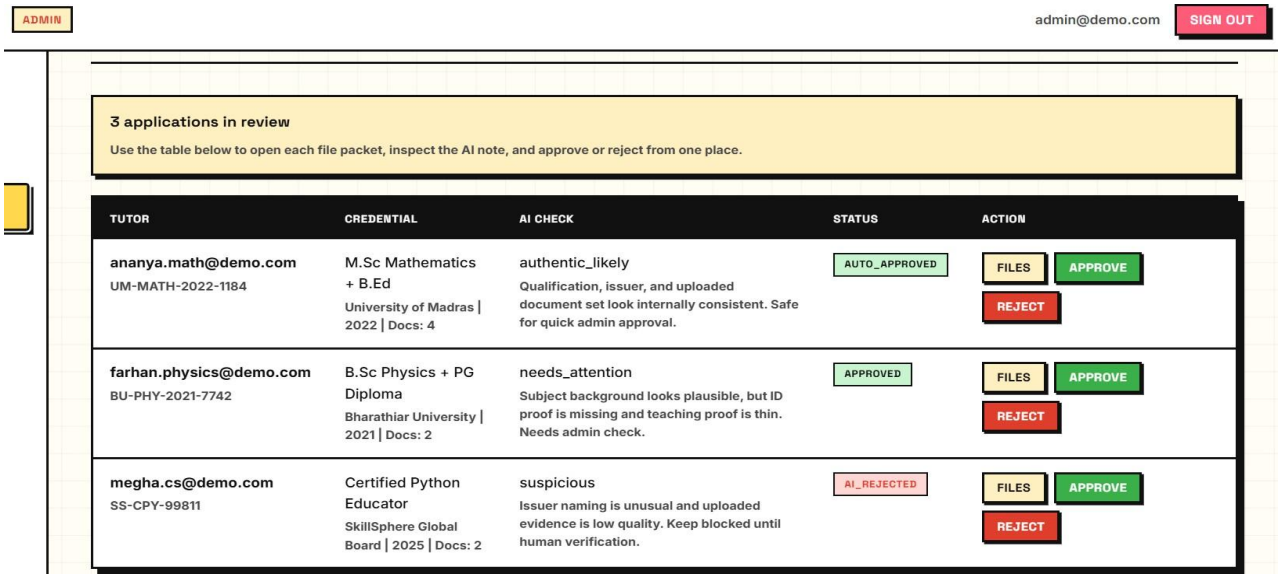


Figure 17. Tutor Approval Automation in Admin profile